

Introduction

**Goal:** Realizzazione di un servizio in Java che fa il calcolo dei valori di una espressione matematica con deployment del servizio in docker usando il framework javalin

Requirements

1. realizzare un servizio Java che calcoli i valori dell'espressione  $\sin(x) + \cos(\sqrt{3} * x)$
2. utilizzare il [javalin server](#) per abilitare la ricezione di messaggi via WebSocket, ma anche HTTP/1 e HTTP2
3. All'utente verrà presentata una pagina html che gli permetta di specificare la x per cui si vuole ottenere il risultato e la modalità di interazione (WebSocket, Get o Post)
4. il deployment dell'applicazione deve avvenire mediante Docker

Requirement analysis

La realizzazione della funzione per calcolare il risultato dell'espressione data in Java è banale.

```
protected double eval(double x) {
    return Math.sin(x) + Math.cos( Math.sqrt(3)*x);
}
```

La complessità sta nella gestione delle richieste di diverso tipo. Sebbene javalin permetta di gestire con codice molto simile sia le connessioni WebSocket che le interazioni HTTP, le due funzionalità sono profondamente diverse:

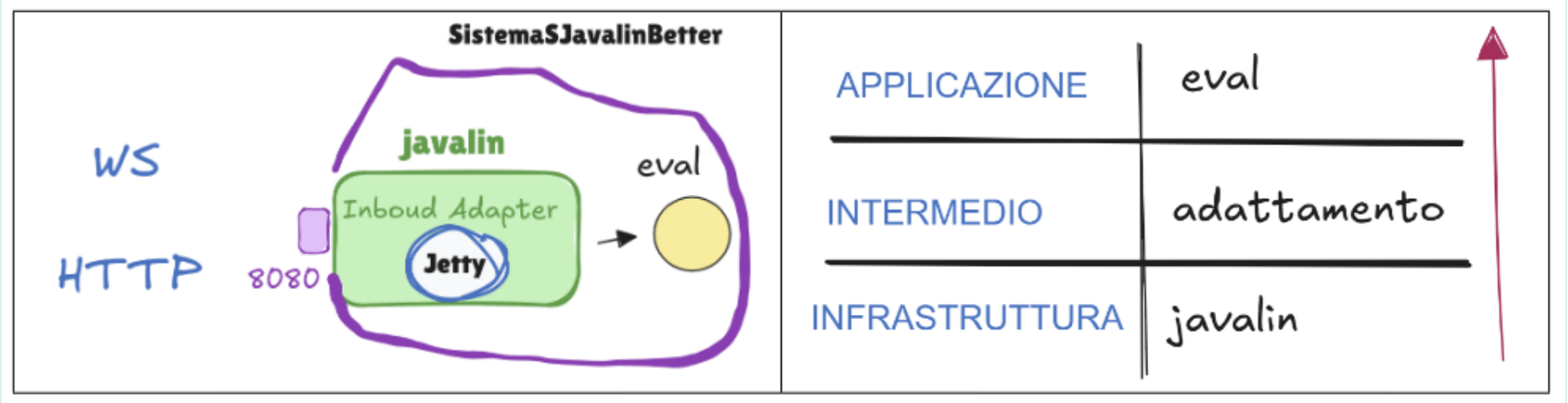
- una WebSocket stabilisce una relazione tra il client e il server che rimane attiva fino a quando il client non chiude la connessione. Da notare che Jetty, su cui si appoggia javalin, chiude automaticamente le connessioni WS se non rileva traffico dopo un certo intervallo di tempo. Per evitare ciò si può agire sul paraemtro Idle Timeout del server o inviare periodicamente dei ping di heartbeat per tenere aperta la connessione
- una interazione HTTP stabilisce invece una transazione che termina quando il server invia il risultato

All'utente verrà presentata la seguente [pagina html](#) che presenta una sezione di input dove l'utente potrà selezioanre al x desiderata, una serie di bottoni per scegliere la modalità di comunicazione e un'area di output dove comparirà il risultato.

Per quanto riguarda la distribuzione mediante una immagine Docker, sono stati realizzati i seguenti file: [sistemaSJavalin.yaml](#) e [Dockerfile](#).

Problem analysis

Sono dipendente dalla tecnologia di javalin, ma cerco di organizzare il codice in maniera opportuna. Sulla base dei principi di Separation of Concerns e Clean Architecture il sistema verrà sviluppato con la seguente infrastruttura:



Il sistema avrà quindi una architettura a Layer:

1. **Livello di infrastruttura:** qui opera javalin, gestendo i protocolli e prelevando dai messaggi le informazioni rilevanti per il livello di dominio. In questo livello è opportuno separare (logicamente) la parte di supporto alle WebSocket e la parte di supporto ad HTTP;
2. **Livello di adattamento:** è il livello intermedio che trasforma i dati di basso livello forniti dall'infrastruttura nei dati richiesti dall'applicazione. Anche qui è opportuno separare le funzioni di utilità per le WebSocket da quelle per HTTP;
3. **Livello di dominio:** la funzione eval vista prima definisce la business logic del sistema, indipendente da javalin e protocolli vari.

Come già detto, javalin gira sopra il Server Jetty. Poiché ogni volta che arriva un messaggio, Jetty assegna il compito di eseguire il codice dentro onMessage a un thread preso dal suo Thread Pool, se arrivano due messaggi quasi contemporaneamente vengono prelevati e sfruttati due thread distinti. Questo significa che se ho una richiesta "lenta" dovuta ad una carico computazionale più alto (simulata con un ritardo) che arriva prima e una più veloce che arriva dopo, non sarà rispettato l'ordine di arrivo nelle risposte, ma arriverà prima la risposta veloce e poi quella lenta. E' importante tenere a mente questo comportamento, perché è in contrasto con quanto avviene normalmente nelle WS, dove ci si aspetta un'esecuzione sequenziale delle richieste ricevute.

Per la gestione delle richieste da parte dell'utente sono necessarie [tre funzioni Javascript](#) collegate ai pulsanti presentati all'utente che vadano ad inviare al server le richieste come specificato.

Per la realizzazione del server è stato realizzato il seguente [codice](#).

Test plans

Project

Testing

Deployment

Maintenance

